

Proceedings of the 2026 ACM Symposium on Autonomous
Systems and Distributed Intelligence (ASADI '26)

ClockChain: A Deterministic Behavioral Ledger for Agent Identity Anchoring, Auditability, and Multi-Agent Coordination

Leon A.¹, Marcus D. Ferreira², Yuki T. Nakashima³, Priya S.
Venkataraman¹, Dmitri O. Reznov⁴

¹Institute for Distributed Intelligence Systems, University of Texas at
Austin ²Department of Computer Science, ETH Zürich ³AI
Governance Laboratory, Keio University ⁴Scalable Systems Group,
Carnegie Mellon University

Received: 12 March 2026 | Accepted: 9 June 2026 | Published: July
21, 2026 | DOI: 10.1145/3692841.3729004

Abstract

Autonomous AI agents operating in distributed enterprise and regulated environments increasingly execute consequential decisions without a standardized mechanism for behavioral auditability, identity persistence, or deterministic state replay. Existing primitives — including blockchain ledgers, append-only event logs, and provenance graphs — address subsets of these requirements but fail to unify them under a coherent, agent-native, compliance-grade protocol. This paper

introduces **ClockChain**, a deterministic behavioral ledger that anchors agent identity to a cryptographically-ordered sequence of behavioral commits called *ClockFrames*. *ClockChain* is designed as a local-first, append-only structure that can federate across agent networks via the *ClockSync* gossip protocol, supporting causal consistency without global consensus overhead. The principal contributions of this work are: (i) a formal model of agent identity anchoring via cryptographically-derived *AgentIDs* bound to a genesis *ClockFrame*; (ii) a deterministic two-phase commit model producing byte-reproducible frame sequences; (iii) a multi-agent synchronization schema based on causal cross-frame references forming a DAG of behavioral dependencies; (iv) formal replayability guarantees including full, partial, and counterfactual replay modes; (v) a compliance mapping to GDPR, MiFID II, SOC 2, and the NIST AI Risk Management Framework; and (vi) an application analysis across five fintech deployment scenarios. Analytical evaluation demonstrates that *ClockChain* imposes $O(1)$ append cost per behavioral commit, approximately 2.1 KB per *ClockFrame*, and 2–5 ms inter-agent synchronization overhead, establishing it as a viable behavioral auditability substrate for high-throughput autonomous agent systems.

Keywords: behavioral auditability, agent identity, deterministic ledger, causal ordering, multi-agent systems, compliance, *ClockChain*, *ClockFrame*, distributed systems

1. Introduction

The proliferation of autonomous AI agents across enterprise, financial, and regulated environments represents one of the most consequential architectural shifts in the history of deployed software systems. Where conventional software executes deterministic instruction sequences under direct human supervision, modern autonomous agents — including large language model (LLM)-based orchestration pipelines, multi-agent trading systems, and autonomous compliance monitors — operate across extended time horizons, interact with dynamic

environments, and make sequences of consequential decisions without real-time human review [1, 2]. This operational profile introduces a category of accountability risk that existing software audit infrastructure was not designed to address.

Behavioral auditability — the capacity of an external party to reconstruct, with full fidelity, what an agent did, in what order, under what authority, and with what state context — has emerged as a first-class engineering requirement in autonomous systems. Regulatory frameworks including the European Union's MiFID II directive [3], the General Data Protection Regulation (GDPR) [4], and the United States NIST AI Risk Management Framework [5] impose explicit obligations for explainability, accountability, and auditability of automated decision-making systems. Safety engineering disciplines, informed by decades of aerospace and industrial control precedent, treat auditability as a prerequisite for certifiability [6]. The practical demands of forensic incident response — reconstructing why an autonomous trading agent triggered a flash-crash event, or why a credit-scoring pipeline issued a discriminatory decision — require not merely a log, but a deterministic, replayable, causally-ordered behavioral history.

Existing systems address fragments of this requirement. Blockchain ledgers [7, 8] provide tamper-evident, cryptographically-ordered records, but impose probabilistic finality, prohibitive throughput overhead, and — critically — carry no native semantics of agent identity or behavioral state. Append-only audit logs [9] are efficient but lack cryptographic ordering guarantees and fragile identity linkage. The W3C PROV provenance model [10] captures agent-action relationships in a graph structure but defines no commit protocol and operates post-hoc rather than in real-time. Decentralized Identifier (DID) specifications [11] provide robust identity primitives but attach no behavioral ledger. The result is a persistent architectural gap: no existing system unifies persistent agent identity, causally-ordered behavioral commits, cryptographic tamper-evidence, deterministic replayability, and compliance-grade auditability in a single, lightweight, deployable protocol.

This paper introduces ***ClockChain***, a novel primitive designed to close this gap. ClockChain is a per-agent, append-only, cryptographically-linked sequence of ***ClockFrames*** — each representing an atomic behavioral commit recording what the agent intended, what it did, and the cryptographic hash of its state before and after

the action. ClockChain is designed to be local-first: a single agent can operate a ClockChain without any network infrastructure, while the ***ClockSync*** protocol enables federation across agent networks when multi-agent causal coordination is required. The design philosophy prioritizes deployability as a software library over adoption as a network protocol, substantially lowering the barrier to integration.

The principal contributions of this work are as follows:

1. **Formal Agent Identity Anchoring Model:** A cryptographic derivation function binding AgentID to a genesis ClockFrame, enabling persistent, verifiable, behavior-linked identity without an external identity server.
2. **Deterministic Commit Model with ClockFrames:** A two-phase behavioral commit protocol producing cryptographically-chained ClockFrames with byte-reproducible determinism guarantees under identical input conditions.
3. **Multi-Agent Synchronization via ClockSync:** A lightweight gossip-based protocol for cross-agent causal dependency tracking forming a globally consistent behavioral DAG without global consensus.
4. **Replayability Guarantees:** Formal definitions and architectural mechanisms for full, partial, and counterfactual replay of agent behavioral histories from ClockVault snapshots.
5. **Compliance Mapping:** A structured mapping of ClockChain properties to the requirements of GDPR, MiFID II, SOC 2, and the NIST AI RMF, demonstrating compliance-grade auditability.
6. **Fintech Application Analysis:** Analysis of five high-value fintech deployment scenarios, with storage and latency overhead estimates.

The remainder of this paper is organized as follows. Section 2 formalizes the Behavioral Auditability Problem and defines the five core requirements ClockChain must satisfy. Section 3 situates ClockChain within the landscape of related work. Section 4 presents the ClockChain architecture. Section 5 defines the deterministic commit model. Section 6 elaborates the agent identity anchoring mechanism. Section 7 describes the ClockSync multi-agent synchronization protocol. Section 8

formalizes replayability. Section 9 maps ClockChain to regulatory compliance frameworks. Section 10 analyzes fintech applications. Section 11 compares ClockChain to blockchain alternatives. Section 12 concludes. Section 13 identifies directions for future work.

2. Problem Statement

2.1 The Behavioral Auditability Problem

In deployed multi-agent systems, the ***Behavioral Auditability Problem*** (BAP) is defined as follows: given a system of n autonomous agents operating over a time horizon T , an external auditor cannot — using available logging and monitoring infrastructure — reconstruct with fidelity what any given agent did, in what order, with what state, under whose authority, or as a consequence of what prior agent actions. This failure is not merely a matter of incomplete logging; it is structural, arising from three independent failure modes that compound in practice.

2.2 Core Failure Modes

(a) Identity Ambiguity. Agents in most deployed systems are identified by session tokens, IP addresses, process IDs, or database row identifiers — none of which constitute persistent, cryptographically-verifiable, behavior-linked identity. An agent restarted after a crash typically acquires a new session identifier, severing continuity with its prior behavioral history. In a multi-agent orchestration pipeline, sub-agents spawned dynamically may share identity credentials with their parent, rendering individual behavioral attribution impossible. Without a persistent identity primitive that survives restarts, delegation chains, and infrastructure changes, behavioral attribution is fundamentally unreliable.

(b) Commit Ordering Ambiguity. Even when individual agent actions are logged, the causal ordering of those actions across a distributed agent system is typically not guaranteed. Distributed system logs are subject to clock skew, network reordering, and buffering delays that can produce apparent orderings inconsistent with true causal precedence [12]. Without a mechanism that enforces causal ordering — such

as Lamport clocks [13] or their behavioral analogs — an auditor attempting to reconstruct a multi-agent decision chain cannot determine whether Agent B's action preceded or followed Agent A's, even when timestamps suggest otherwise. The consequences for forensic accuracy are severe.

(c) Replay Impossibility. Perhaps the most consequential failure mode is the impossibility of deterministic replay. Conventional logs record what happened but not the precise state context required to re-execute the decision. External inputs — market data feeds, database reads, API responses — are rarely captured in their exact form at the moment of decision. Agent internal state at decision time is typically not snapshotted. The result is that re-running an agent on a log-derived input sequence produces a different behavioral trajectory, making forensic replay non-deterministic and scientifically inadmissible as evidence of what actually occurred.

2.3 Formal Requirements

These failure modes motivate the following formal requirements for a behavioral auditability system. A compliant system must satisfy all five:

- **R1 — Identity Persistence:** Agent identity must persist across session boundaries, restarts, infrastructure changes, and delegation chains; must be cryptographically verifiable by third parties without access to identity servers.
- **R2 — Causal Ordering:** All behavioral commits must carry a causally-correct ordering guarantee; the ordering must be globally consistent across agent boundaries without requiring global consensus.
- **R3 — Deterministic Replayability:** Given a complete behavioral record, any authorized replay engine must be able to reconstruct the agent's full decision history and re-execute any suffix of its behavioral chain, producing state-identical outputs under equivalent input conditions.
- **R4 — Multi-Agent Auditability:** Cross-agent causal dependencies must be captured in a form that enables auditors to reconstruct the full causal graph

of a multi-agent decision, attributing each sub-decision to its originating agent with full behavioral context.

- **R5 — Regulatory Compliance:** The behavioral record must satisfy the evidentiary and explanatory requirements of applicable regulatory frameworks, including GDPR Article 22 (right to explanation), MiFID II Articles 26–27 (transaction reporting and best execution), SOC 2 Trust Service Criteria, and the NIST AI RMF.

2.4 Motivating Scenario

Consider a fintech firm operating a multi-agent system responsible for autonomous execution of equity trades on behalf of institutional clients. The system comprises a market observation agent, a strategy selection agent, a risk assessment agent, and an order execution agent — each operating concurrently and exchanging signals through a shared message bus. On a given trading day, a sequence of rapid automated orders triggers a price dislocation event, and the firm's regulators — acting under MiFID II Article 26 obligations — demand a complete audit of the decision chain: which agent decided to enter a position, based on what market state, following what signals from upstream agents, subject to what risk constraints, and at what exact causal sequence of events? With conventional logging infrastructure, this demand cannot be satisfied. ClockChain is designed precisely to make it satisfiable.

3. Background and Related Work

3.1 Causal Ordering in Distributed Systems

The foundational treatment of causal ordering in distributed systems is due to Lamport [13], whose logical clock assigns scalar timestamps to events such that if event a causally precedes event b , then $\text{clock}(a) < \text{clock}(b)$. Fidge [14] extended this to vector clocks, where each process maintains a vector of counters capturing the causal history of all processes — enabling detection of concurrent events. Both mechanisms establish the theoretical substrate for causal ordering in distributed systems. However, neither attaches behavioral semantics to the ordered events: a

Lamport clock records that event a preceded event b , but records nothing about what a and b were, or what agent state they transformed. ClockChain extends causal ordering into the behavioral domain by embedding it within semantically-rich ClockFrames that capture action type, state hashes, policy bindings, and agent identity.

Kulkarni et al. [15] proposed Hybrid Logical Clocks (HLC), which combine the causality-capture of logical clocks with the wall-clock sortability of physical timestamps. HLC addresses the practical limitation that pure logical clocks cannot be correlated with real-world time — a significant liability for regulatory reporting. ClockChain adopts HLC as its primary ordering mechanism, inheriting both causal correctness and regulatory-grade temporal correlation.

3.2 Blockchain and Distributed Ledgers

Nakamoto [7] introduced Bitcoin's proof-of-work blockchain, establishing the paradigm of a tamper-evident, append-only, distributed ledger with probabilistic finality. Buterin [8] and Wood [16] extended this to programmable computation via Ethereum's smart contract platform. Both systems provide powerful tamper-resistance through cryptographic chaining and decentralized consensus, but they impose fundamental design constraints that render them unsuitable as behavioral auditability substrates for autonomous agents: (i) finality is probabilistic and latency is high (seconds to minutes), incompatible with near-real-time behavioral commit requirements; (ii) throughput is limited (typically 10–100 transactions per second for public chains), while autonomous agents may commit hundreds of behavioral frames per minute; (iii) no native agent identity primitive exists — blockchain addresses are key-pairs with no behavioral semantics attached; and (iv) deployment requires network participation, substantially increasing operational complexity. ClockChain is explicitly designed to address all four limitations.

3.3 Audit Logging Systems

Schneier and Kelsey [9] analyzed secure audit logging systems and identified the fundamental requirements for tamper-evidence: append-only structure, cryptographic chaining, and secure key management. Their work established the theoretical basis for hash-chained logs but did not address behavioral semantics,

agent identity, or replay capabilities. Conventional append-only log systems (e.g., Apache Kafka, AWS CloudTrail) satisfy some of these properties in practice but lack cryptographic per-entry signatures, do not enforce causal ordering across distributed writers, and capture no state context — failing requirements R1, R2, and R3.

3.4 Provenance and Agent Models

The W3C PROV model [10] provides a standardized vocabulary for capturing provenance — the origin and history of data and processes — using entities, activities, and agents as first-class constructs. PROV captures rich semantic relationships, including derivation, attribution, and delegation, and is well-suited for post-hoc provenance reconstruction. However, PROV defines no real-time commit protocol, provides no cryptographic ordering guarantee, and does not address replayability or identity persistence. Wooldridge and Jennings [17] provide a foundational formal model of intelligent agents, including belief-desire-intention (BDI) architecture, but do not address auditability infrastructure. ClockChain can be viewed as a provenance-compatible commit protocol that operationalizes PROV semantics in real-time with cryptographic guarantees.

3.5 Decentralized Identity and Agent Identity

The W3C Decentralized Identifier (DID) specification [11] defines a standard for self-sovereign identifiers that do not depend on centralized identity providers. DIDs are cryptographically-controlled, independently verifiable, and support delegation. However, DIDs are identity-first and behavior-blind: they establish who an agent is but capture nothing of what the agent has done. ClockChain's AgentID mechanism is DID-compatible — an AgentID can be expressed as a DID — but extends the identity primitive to be behavior-first by deriving identity from the agent's genesis behavioral commitment.

3.6 Recent Work on Auditable Agents

Recent work has begun to address runtime auditability for LLM-based agents [18, 19]. Chen et al. [18] propose a trace-capture mechanism for LLM tool-call sequences, enabling post-hoc review of agent actions in customer service applications. Vaziry et

al. [20] analyze economic identity for agent-to-agent (A2A) protocols, proposing ledger-anchored identity tokens for financial agent coordination. While these contributions advance the state of practice, they do not provide a unified protocol combining identity anchoring, causal ordering, and deterministic replayability. ClockChain synthesizes and formalizes these directions into a coherent, specification-grade framework.

3.7 Gap Synthesis

No existing system simultaneously satisfies requirements R1-R5 as defined in Section 2.3. Table 1 summarizes the coverage of related approaches. ClockChain is positioned to address the full requirement set by design.

System / Approach	R1 Identity	R2 Ordering	R3 Replay	R4 Multi-Agent	R5 Compliance
Lamport / Vector Clocks [13, 14]	✗	✓	✗	Partial	✗
Blockchain [7, 8]	✗	✓	✗	✓	Partial
Audit Logs [9]	Partial	✗	✗	✗	Partial
W3C PROV [10]	Partial	✗	✗	✓	Partial
W3C DID [11]	✓	✗	✗	✗	Partial
Auditable Agents [18, 19]	Partial	Partial	✗	✗	Partial
ClockChain (this work)	✓	✓	✓	✓	✓

Table 1. Coverage of requirements R1-R5 across related systems. ✓ = satisfied, ✗ = not satisfied, Partial = partially satisfied.

4. ClockChain Architecture

4.1 Overview

ClockChain is a per-agent, append-only, cryptographically-linked sequence of ClockFrames. Each ClockFrame represents a single atomic behavioral commit — an indivisible record of one unit of agent behavior, sealed with a cryptographic signature and chained to its predecessor by hash. The structure is analogous to a blockchain in its append-only, hash-chained form, but differs fundamentally in scope (per-agent rather than network-global), semantics (behavioral rather than transactional), and consensus model (local determinism rather than distributed agreement).

A ClockChain instance is associated with exactly one logical agent and begins with a distinguished ***ChainRoot*** (genesis ClockFrame). It grows monotonically as the agent commits behavioral events. The chain is stored in a ***ClockVault*** — an append-only persistence layer whose implementation may vary by deployment tier. An ***Inspector Interface*** provides read-only traversal for auditors, regulators, and orchestrating agents.

4.2 Core Components

4.2.1 AgentID

The ***AgentID*** is a 32-byte persistent identity token derived cryptographically from the agent's Ed25519 public key [21], the hash of its genesis ClockFrame, and the hash of its capability manifest. The derivation function uses HKDF [22] to ensure that the AgentID is unique, unforgeable, and binding to the agent's initial behavioral commitment. Critically, the AgentID is stable across session restarts: any authorized reinstatement of an agent with the same key material and genesis frame will derive an identical AgentID, enabling identity continuity without an external identity server.

4.2.2 ClockFrame

The ***ClockFrame*** is the atomic unit of commitment in ClockChain. Its structure is defined formally in Section 5. Each ClockFrame contains the agent's identity, a

monotonic logical clock value, a reference to the preceding frame's hash, a semantic description of the action taken, cryptographic hashes of agent state before and after the action, references to applicable compliance policies, and an Ed25519 signature over all preceding fields. The integrity of the chain is established by the invariant that every ClockFrame's *parent_hash* field must equal the SHA3-256 hash of the preceding frame's complete serialized form.

4.2.3 ChainRoot

The ***ChainRoot*** is the genesis ClockFrame — index 0 of every ClockChain. Its *parent_hash* field contains the all-zeros value by convention. The ChainRoot carries the agent's complete capability manifest (a structured declaration of what actions the agent is authorized to perform), its initial policy bindings, and a timestamp-anchored genesis signature. The ChainRoot serves as the trust anchor for all subsequent frames: chain validation always begins by verifying the ChainRoot's signature against the known public key.

4.2.4 ClockVault

The ***ClockVault*** is the persistence layer for ClockFrame sequences. ClockChain defines a logical interface for the ClockVault but allows three physical implementations, corresponding to the three trust tiers described in Section 4.3: (i) a local append-only file or embedded key-value store (e.g., RocksDB) for single-agent deployments; (ii) a distributed, replicated key-value store (e.g., etcd, CockroachDB) for enterprise multi-agent deployments; and (iii) a federated consensus store with optional external hash checkpointing for regulatory-grade deployments. The ClockVault exposes four operations: *append(frame)*, *read(index)*, *validate(from, to)*, and *snapshot()*.

4.2.5 Inspector Interface

The ***Inspector Interface*** is a read-only, authenticated API through which auditors, regulators, and orchestrating agents traverse a ClockChain. It exposes frame iteration, range queries by logical clock index, cross-reference resolution (for ClockSync dependencies), and full-chain cryptographic validation. The Inspector Interface enforces access control via capability tokens derived from the AgentID delegation chain — ensuring that read access is itself auditable and non-repudiable.

4.3 Trust Tiers

ClockChain defines three deployment trust tiers, reflecting the escalating tamper-resistance requirements of different operational contexts:

- **Tier 1 — Local Self-Signed:** The ClockVault is a local file or embedded store; frames are signed by the agent's own key; suitable for single-agent debugging, development, and testing. Tamper-evidence is limited to the integrity of the local storage medium.
- **Tier 2 — Federated Validator Set:** The ClockVault is replicated across a set of federated validators (minimum 3, recommended 5); frame acceptance requires a quorum of validator acknowledgments; suitable for enterprise multi-agent deployments where a trusted operator controls the validator set. Tamper-evidence is Byzantine fault tolerant [23] against a minority of malicious validators.
- **Tier 3 — External Hash Anchoring:** Periodic Merkle roots of the ClockVault are anchored to an external, publicly-auditable hash registry (which may be a public blockchain, a regulatory submission system, or a trusted timestamping authority per RFC 3161); this tier satisfies evidentiary-grade tamper-evidence for regulatory and legal proceedings.

Figure 1: ClockChain Architecture — Three-Tier Trust Model

Tier 1 (Local): Agent → ClockFrame Commit → Local ClockVault (append-only file / RocksDB). Single-agent self-verification. Ed25519 signature per frame. SHA3-256 chain integrity.

Tier 2 (Federated): Agent → ClockFrame Commit → Federated ClockVault (etcd / CockroachDB cluster). Frame broadcast to validator set. Quorum acknowledgment required. ClockSync DAG integration. Inspector Interface exposed to orchestrators.

Tier 3 (Anchored): Tier 2 infrastructure + periodic Merkle root computation over ClockVault segments → submission to external hash registry (public blockchain checkpoint / RFC 3161 timestamp authority / regulatory submission endpoint). Full-chain validation traversable by any third party with Inspector Interface access.

All tiers share the same ClockFrame schema, AgentID derivation function, and Inspector Interface specification. Tier selection is a deployment decision, not a protocol decision.

4.4 Security Properties

Collision Resistance: The SHA3-256 hash function [24] is used for all parent_hash chaining operations. SHA3-256 provides 128 bits of collision resistance, sufficient for all anticipated deployment lifetimes. The NIST-standardized Keccak construction provides resistance against length-extension attacks that affected SHA-2 family functions.

Non-Repudiation: Every ClockFrame carries an Ed25519 signature [21] over the complete frame contents, computed using the agent's private signing key. Ed25519 provides 128-bit security with compact 64-byte signatures and fast verification, making per-frame signing computationally negligible. Signature verification requires only the agent's public key, which is embedded in the ChainRoot and derivable from the AgentID.

Tamper Detection: Chain validation traverses the complete ClockFrame sequence, verifying for each frame that (i) its parent_hash matches the SHA3-256 of the preceding frame, (ii) its signature verifies against the agent's public key, (iii) its frame_index is exactly one greater than its predecessor, and (iv) its monotonic_ts is strictly greater than its predecessor. Any violation of these invariants constitutes a tamper event and raises a *ChainViolation* exception.

4.5 Performance Characteristics

ClockChain's core operations exhibit the following computational complexities: (i) *append*: $O(1)$ — a single SHA3-256 hash computation and Ed25519 signature; (ii) *sequential traversal*: $O(n)$ — linear scan of n frames; (iii) *Merkle-path verification*: $O(\log n)$ — a standard Merkle audit path computation over the ClockVault's frame Merkle tree. These characteristics ensure that ClockChain does not impose superlinear overhead as behavioral histories grow, making it suitable for long-lived agents executing high volumes of behavioral commits.

5. The Deterministic Commit Model

5.1 Behavioral Commits

A ***behavioral commit*** in ClockChain is a two-phase operation that atomically records a single unit of agent behavior. The two-phase structure — Pre-Commit followed by Post-Commit — mirrors the logical structure of agent action: an agent first declares its intent and binds it to the current world state, then executes the action and records the outcome. This two-phase design is essential for replay fidelity: it ensures that both the intended action and the actual outcome are captured, enabling retrospective analysis of divergences between intent and execution.

Phase 1 — Pre-Commit: The agent computes the hash of its current internal state (`pre_state_hash`), declares its intended action type and payload (`action_type`, `action_payload`), and records the current HLC timestamp. These fields, along with the `parent_hash` and `agent_id`, are written to a pending frame record. The pending frame is not yet sealed or signed.

Phase 2 — Post-Commit: Following execution of the declared action, the agent computes the hash of its post-execution internal state (`post_state_hash`), appends this to the pending frame record, and seals the frame with an Ed25519 signature over all fields. The sealed frame is then appended to the ClockVault. From this point, the frame is immutable.

5.2 ClockFrame Schema

```
ClockFrame {  frame_index      : uint64      // Monotonically increasing frame
sequence number  parent_hash    : bytes[32]  // SHA3-256 of the complete
serialization of frame[index-1]              // All-zeros for ChainRoot
(frame_index == 0) agent_id      : bytes[32]  // Persistent AgentID (see Section
6)  monotonic_ts    : int64      // Hybrid Logical Clock value [Kulkarni et al., 2014]
wall_clock_ts     : RFC3339      // Wall-clock timestamp (informational; not used for
ordering)  action_type    : enum {              OBSERVE, // Agent observed
external state          DECIDE, // Agent made an internal decision
ACT, // Agent executed an external action          COMMUNICATE, //
Agent sent a message to another agent          TERMINATE // Agent
terminated or was revoked          }  action_payload : bytes      //
Serialized action descriptor (agent-defined schema)  pre_state_hash  : bytes[32]
// SHA3-256 of agent's complete internal state before action  post_state_hash  :
bytes[32] // SHA3-256 of agent's complete internal state after action  policy_refs
: []string // List of compliance policy identifiers bound to this frame
// e.g., ["GDPR-Art22", "MiFID2-Art27", "SOC2-CC7.2"]  causal_deps    : []{
// Cross-agent causal dependencies (ClockSync; see Section 7)
agent_id : bytes[32]          frame_index: uint64          }
signature : bytes[64] // Ed25519 signature over SHA3-256(all fields above) }
```

5.3 Determinism Guarantee

ClockChain's determinism guarantee states: given the same ChainRoot (identical capability manifest, policy bindings, and genesis signature) and the same input sequence (identical pre_state_hash values for each frame in order), a conforming ClockChain implementation will produce byte-identical ClockFrame sequences. This guarantee holds under the assumption that the agent's logic is itself deterministic —

that is, given identical state and input, the agent produces identical output. For LLM-based agents, determinism requires fixed random seeds and temperature-zero sampling; for symbolic agents, determinism is typically inherent.

The determinism guarantee is the formal basis for the replayability claims established in Section 8. It is important to note that the guarantee is conditional, not unconditional: external side effects (network calls, database reads, real-time data feeds) are captured as action descriptors in `action_payload` but are not themselves deterministically reproducible without shadow-mode mocking. Section 8 addresses this boundary condition in detail.

5.4 Hybrid Logical Clock Ordering

ClockChain uses Hybrid Logical Clocks (HLC) [15] for the `monotonic_ts` field. HLC combines a logical counter with a wall-clock component, producing timestamps that satisfy both causal ordering (if event a causally precedes event b , then $HLC(a) < HLC(b)$) and approximate wall-clock correlation (HLC values are within a bounded skew of physical time, typically less than the maximum clock skew in the deployment environment). This dual property is essential for ClockChain: causal ordering satisfies R2, while wall-clock correlation enables regulatory reporting formats that require real-world timestamps (satisfying R5).

5.5 Immutability and Corrective Commits

ClockChain frames are strictly immutable once sealed. This is a fundamental design constraint: mutability would undermine the tamper-detection properties of the hash chain. When an agent discovers an error in a prior commit — for example, an incorrectly recorded `action_payload` — it does not modify the erroneous frame. Instead, it issues a ***corrective commit***: a new frame of type `DECIDE` or `ACT` that references the erroneous frame's index in its `action_payload` and describes the nature and correction of the divergence. Corrective commits are fully auditable, preserving the complete history of both the error and its correction.

5.6 Algorithm 1: ClockFrame Commit Protocol

Algorithm 1: ClockFrame Commit Protocol

Input: $action_type \in \{\text{OBSERVE, DECIDE, ACT, COMMUNICATE, TERMINATE}\}$ $action_payload : \text{bytes}$ (serialized action descriptor)
 $policy_refs : []\text{string}$ (applicable compliance policy IDs) $causal_deps : []\{\text{agent_id, frame_index}\}$ (optional cross-agent deps) Output: $sealed_frame : \text{ClockFrame}$ (immutable, appended to ClockVault) Preconditions: - Agent holds a valid private signing key K_{priv} - ClockVault is accessible and $last_frame = \text{ClockVault.last}()$ - Agent internal state S is fully defined and serializable

Phase 1 — PRE-COMMIT:

1. $prev_hash \leftarrow \text{SHA3-256}(\text{serialize}(last_frame))$
// All-zeros if ClockVault is empty (ChainRoot)
2. $idx \leftarrow last_frame.frame_index + 1$
3. $ts_hlc \leftarrow \text{HLC.now}()$ // Hybrid Logical Clock tick
4. $ts_wall \leftarrow \text{system.wall_clock}()$ // RFC3339 wall time (informational)
5. $pre_hash \leftarrow \text{SHA3-256}(\text{serialize}(S))$ // Hash current agent state
6. $pending \leftarrow \{$
 $frame_index = idx,$
 $parent_hash = prev_hash,$ $agent_id = \text{AgentID},$
 $monotonic_ts = ts_hlc,$ $wall_clock_ts = ts_wall,$
 $action_type = action_type,$ $action_payload = action_payload,$
 $pre_state_hash = pre_hash,$ $post_state_hash = \emptyset,$ // Not yet computed
 $policy_refs = policy_refs,$ $causal_deps = causal_deps,$
 $signature = \emptyset$ // Not yet computed
 $\}$

EXECUTION PHASE:

7. $result \leftarrow \text{execute}(action_type, action_payload)$ // Perform the declared action; capture outcome

Phase 2 — POST-COMMIT:

8. $S' \leftarrow \text{update_state}(S, result)$ // Apply action outcome to state
9. $post_hash \leftarrow \text{SHA3-256}(\text{serialize}(S'))$ // Hash updated agent state
10. $pending.post_state_hash \leftarrow post_hash$
11. $frame_body \leftarrow \text{serialize}(\{$
 $pending.frame_index, pending.parent_hash,$
 $pending.agent_id,$ $pending.monotonic_ts, pending.wall_clock_ts,$
 $pending.action_type, pending.action_payload,$
 $\})$

```

pending.pre_state_hash, pending.post_state_hash,
pending.policy_refs, pending.causal_deps      }) 12. sig      ←
Ed25519.Sign(K_priv, SHA3-256(frame_body)) 13. pending.signature ← sig 14.
sealed_frame ← pending                        // Frame is now sealed 15.
ClockVault.append(sealed_frame)                // Append to persistent store 16.
HLC.update(ts_hlc)                            // Advance HLC for next commit 17. S ← S'
// Update agent state reference 18. return sealed_frame
Error Handling: - If
execute() raises an exception: create sealed_frame with action_type = DECIDE,
action_payload = {error: exception_message}, post_state_hash = pre_hash
(state unchanged), then proceed from step 10. - If ClockVault.append() fails: retry
with exponential backoff; if persistent failure, raise ChainVaultError (agent must
halt until vault is restored). Invariant Maintained:  $\forall i > 0: \text{frames}[i].\text{parent\_hash} =$ 
 $\text{SHA3-256}(\text{serialize}(\text{frames}[i-1]))$   $\forall i > 0: \text{frames}[i].\text{monotonic\_ts} > \text{frames}[i-$ 
 $1].\text{monotonic\_ts}$   $\forall i: \text{Ed25519.Verify}(\text{AgentID}.\text{public\_key}, \text{frames}[i].\text{signature}) =$ 
true

```

6. Agent Identity Anchoring

6.1 The Identity Anchoring Problem

Agent identity in the ClockChain context must satisfy four properties that collectively define ***behavioral identity anchoring***: (a) *persistence* — the identity must remain stable across session restarts, crashes, and infrastructure migrations; (b) *unforgeability* — no party without access to the agent's private key material can generate a valid AgentID or produce frames that verify as originating from that agent; (c) *independent verifiability* — any third party with access to the agent's public key and genesis ClockFrame can verify the AgentID without contacting an identity server; and (d) *behavior-linkage* — the identity must be cryptographically

bound to the agent's first behavioral commitment, ensuring that an agent's identity is inseparable from its behavioral history.

6.2 AgentID Derivation Function

The AgentID is derived using the HKDF [22] key derivation function as follows:

```
AgentID = HKDF-SHA3-256(IKM = public_key || genesis_frame_hash ||  
capability_manifest_hash, salt = "clockchain-v1-agentid", info =  
deployment_context_id)
```

where *public_key* is the agent's Ed25519 public key (32 bytes), *genesis_frame_hash* is the SHA3-256 hash of the serialized ChainRoot frame (32 bytes), *capability_manifest_hash* is the SHA3-256 hash of the agent's capability manifest document (32 bytes), and *deployment_context_id* is an optional deployment-specific context string (e.g., an organization identifier or environment label) that prevents AgentID collisions across deployments.

The resulting AgentID is 32 bytes. Its security derives from the preimage resistance of SHA3-256 and the pseudorandomness of HKDF: an adversary cannot derive the underlying key material from the AgentID, cannot forge an AgentID without knowing the private key (since the *genesis_frame_hash* includes an Ed25519 signature), and cannot produce a valid AgentID for a different capability manifest without breaking SHA3-256 collision resistance.

6.3 Genesis Frame and Identity Establishment

An agent establishes its ClockChain identity by constructing and committing its **Genesis Frame** (ChainRoot) as the first act of its operational lifetime. The Genesis Frame contains:

- The agent's complete *capability manifest*: a structured document (JSON-LD schema recommended) declaring the full set of action types the agent is authorized to perform, the resource scopes it may access, and the maximum authority level of any delegated sub-agent it may spawn.

- The agent's initial *policy bindings*: references to the compliance policy identifiers applicable to all subsequent frames unless overridden.
- A *genesis timestamp*: the HLC value at creation, anchored to a trusted time source if Tier 3 compliance is required.
- The Ed25519 public key in the `action_payload` field, enabling key recovery from the chain alone.

The Genesis Frame's `parent_hash` field is set to all-zeros, marking it as the chain origin. Its signature is computed using the agent's private key, establishing the cryptographic trust anchor for all subsequent frames.

6.4 Identity Continuity Across Restarts

When a ClockChain agent restarts — whether due to a planned shutdown, an infrastructure failure, or a container restart — it re-derives its AgentID from its persisted key material and validates this derived AgentID against the ChainRoot stored in the ClockVault. If validation succeeds, the agent resumes appending frames to the existing chain; the chain is continuous and the AgentID is unchanged. If validation fails — indicating either key material corruption or a tampered ChainRoot — the agent raises an IdentityViolation exception and halts pending human intervention.

This mechanism satisfies requirement R1 (Identity Persistence) without requiring any external identity server, session token, or network lookup. The ClockVault itself is the identity authority.

6.5 Identity Delegation

ClockChain supports hierarchical identity delegation for orchestrated multi-agent systems. A parent agent may authorize a child agent by issuing a ***DelegatedAgentID*** through a delegation ClockFrame — a COMMUNICATE-type frame whose `action_payload` contains:

- The child agent's public key and `genesis_frame_hash` (computed by the child prior to delegation)

- A constrained capability subset — a subset of the parent's capability manifest that the child is authorized to exercise
- An optional expiry (expressed as a maximum frame count or HLC bound)
- The parent's signature over these fields

The child's AgentID incorporates the parent's AgentID in its HKDF derivation, creating a cryptographic delegation chain that auditors can traverse to verify the full authority lineage of any child agent's actions. This mechanism supports arbitrarily deep delegation trees while maintaining auditability at every level.

6.6 Identity Revocation

When an agent's operational lifetime ends — planned retirement, security compromise, or policy-driven termination — the final ClockFrame committed is of type TERMINATE. The TERMINATE frame seals the chain: no subsequent frames with the same AgentID are accepted by conforming ClockVault implementations. The TERMINATE frame's action_payload contains a *revocation digest*: a structured record of the reason for termination, the terminating authority's identity, and an optional forwarding reference to a successor AgentID.

Conforming ClockSync peers that receive a TERMINATE frame propagate a *revocation notice* through the gossip network, ensuring that all federated validators reject subsequent frames referencing the revoked AgentID within bounded propagation time.

6.7 Comparison with DID Methods

W3C DID methods [11] provide robust, self-sovereign identity primitives with broad ecosystem support. ClockChain identity is DID-compatible: an AgentID can be expressed as a DID (e.g., `did:clockchain:<base58-encoded-agentid>`), and the ClockVault can serve as a DID document provider via the DID resolution protocol. However, ClockChain identity differs from conventional DID methods in three respects: (i) ClockChain identity is *behavior-first* — the genesis behavioral commitment is part of the identity derivation, not an afterthought; (ii) ClockChain provides a native behavioral ledger, whereas DID methods require external behavioral record systems; and (iii) ClockChain identity does not require DID

resolution infrastructure, reducing operational dependencies for isolated deployments.

7. Multi-Agent Synchronization: The ClockSync Protocol

7.1 The Multi-Agent Causal Coordination Problem

In a multi-agent system, agents do not operate in isolation: Agent B's decision to execute an order may depend causally on Agent A's prior commitment to a market observation. If this causal dependency is not captured in the behavioral record, an auditor cannot reconstruct the true decision graph — producing a forensically incomplete picture that may fail regulatory scrutiny and safety analysis alike. The challenge is to capture cross-agent causal dependencies in a way that is efficient (not requiring global consensus), resilient (tolerating agent failures and network partitions), and auditable (permanently recorded in each agent's ClockChain).

7.2 ClockSync Protocol Overview

The ***ClockSync*** protocol is a lightweight gossip-based protocol that enables agents to declare and verify cross-agent causal dependencies without requiring synchronous inter-agent coordination. ClockSync operates at the frame level: an agent that declares a dependency on another agent's committed frame includes a *causal_deps* entry in its own ClockFrame referencing the depended-upon agent's AgentID and frame_index. This declaration transforms the collection of individual ClockChains into a globally-consistent behavioral DAG.

7.3 Cross-Frame References and the Synchronization DAG

A ***cross-frame reference*** is a tuple (agent_id_A, frame_index_j) appearing in the causal_deps field of a ClockFrame committed by Agent B. This reference asserts: "this action of Agent B causally depended on frame j of Agent A — that is, Agent B had observed or was responding to the state of Agent A as recorded in frame j."

Cross-frame references are unidirectional and append-only: once a dependency is declared, it is immutable.

The collection of all ClockFrames across all agents, connected by their causal_deps cross-frame references, forms a directed acyclic graph (DAG) — the **Multi-Agent Synchronization DAG**. Each node in the DAG is a ClockFrame; each edge represents a causal dependency. The DAG captures the full causal history of a multi-agent system's behavior, enabling auditors to answer questions of the form: "what was the complete causal ancestry of Agent C's frame 42?" by traversing DAG ancestors.

Figure 2: Multi-Agent Synchronization DAG — Example

Agents: *A (Market Observer), B (Strategy Selector), C (Risk Assessor), D (Order Executor).* Frames shown as $A[n]$, $B[n]$, $C[n]$, $D[n]$ for frame index n .

Causal edges (cross-frame references):

$B[3] \rightarrow A[7]$ (*B's strategy decision depends on A's market observation at frame 7*)

$C[2] \rightarrow A[7], B[3]$ (*C's risk assessment depends on both A's observation and B's strategy*)

$D[1] \rightarrow C[2]$ (*D's order execution depends on C's risk clearance*)

$D[1] \rightarrow B[3]$ (*D also directly depends on B's strategy frame*)

DAG structure: $A[7]$ is a common ancestor of $B[3]$, $C[2]$, and $D[1]$. The DAG has no cycles (guaranteed by monotonic frame indices and forward-only reference semantics). Traversal from $D[1]$ to its full causal ancestry yields the complete decision trace for a single order execution event.

Auditor query: "Why did D place order at $D[1]$?" $\rightarrow$ Traverse DAG ancestors $\rightarrow C[2]$ (risk approved), $B[3]$ (strategy: buy), $A[7]$ (market observation: bid-ask

spread < threshold). Full causal chain reconstructed deterministically.

7.4 ClockSync Protocol Steps

The ClockSync protocol executes in three phases for each frame that declares cross-agent dependencies:

Step 1 — Broadcast: Upon committing a ClockFrame containing one or more `causal_deps` entries, the agent serializes the frame's header (`frame_index`, `agent_id`, SHA3-256 hash of the complete frame) and broadcasts this summary to all ClockSync peers registered in its federation configuration. Broadcast is performed over the federation's gossip network (an epidemic gossip protocol with configurable fanout, default fanout = 3).

Step 2 — Echo: Each receiving peer validates that the referenced dependency frames (as declared in `causal_deps`) exist in the referenced agent's ClockVault and are themselves settled. The peer then sends an acknowledgment (echo) containing its own AgentID, the received frame's hash, and its current HLC timestamp. This echo is also propagated through the gossip network to additional peers.

Step 3 — Settle: The originating agent monitors incoming echoes. When a quorum of peers (configurable, default: $\text{floor}(n/2) + 1$ for n validators) have acknowledged receipt of the frame's cross-reference summary, the frame's causal dependencies are marked as *settled*. Settled status is recorded in the ClockVault as a metadata annotation to the frame. An auditor querying a settled frame's dependencies can be confident that the referenced frames exist and have been independently verified by the validator quorum.

7.5 Consistency Model

ClockChain uses *causal consistency* [25] — not strong (linearizable) consistency — as its multi-agent synchronization model. Under causal consistency, agents are guaranteed to observe causally-related events in their correct causal order, but may observe concurrent (causally-unrelated) events in different orders. This choice is

deliberate: strong consistency would require synchronous cross-agent coordination blocking agents on remote commits, introducing unacceptable latency for high-frequency behavioral commit scenarios (e.g., a trading agent committing 1,000 frames per hour). Causal consistency provides audit-grade ordering guarantees without blocking agents on remote acknowledgments unless explicitly required by policy.

When strong ordering is required by policy (e.g., for a regulated multi-agent decision that requires all agents to commit before any agent acts), agents can declare a *synchronization barrier* — a COMMUNICATE-type frame that all participating agents must acknowledge before any agent proceeds. Synchronization barriers are optional and policy-driven, preserving the default non-blocking behavior for the majority of operational scenarios.

7.6 Conflict Detection and Resolution

A ***concurrent commit event*** occurs when two agents commit frames that reference each other without causal ordering — that is, A's frame *j* references B's frame *k*, and B's frame *k* references A's frame *j*. This structural cycle is physically impossible within a single ClockChain (guaranteed by monotonic frame indices), but can arise in cross-agent DAG references when two agents simultaneously declare mutual dependencies.

ClockSync detects concurrent commit events during the Echo phase: a validator that observes a dependency cycle raises a *ConcurrentCommit* event, which is propagated to all peers. Resolution follows one of two policies: (i) *Logical Clock Resolution* — the frame with the lower HLC timestamp is designated as causally prior; (ii) *Orchestrator Intervention* — a designated orchestrator agent commits a DECIDE-type frame that explicitly orders the concurrent frames, creating an authoritative causal linearization. The resolution frame is itself recorded in the orchestrator's ClockChain, making the resolution process fully auditable.

7.7 Latency Characteristics

ClockSync adds an estimated 2–5 ms of overhead per settled inter-agent dependency under typical enterprise LAN conditions (sub-1 ms peer-to-peer latency, 3–5 validator

peers). This overhead is incurred only for frames that declare cross-agent dependencies; frames with no `causal_deps` entries (the majority in typical deployments) incur no ClockSync overhead. The 2-5 ms settlement latency is suitable for near-real-time coordination in financial, operational, and compliance-monitoring applications. Applications requiring sub-millisecond inter-agent coordination (e.g., co-located high-frequency trading) should route inter-agent signals through the agent logic layer rather than ClockSync, with ClockSync used for post-hoc auditability rather than real-time coordination gating.

7.8 Application to LLM-Based Orchestration

In LLM-based agent orchestration frameworks [18, 19], tool calls, sub-agent invocations, and context handoffs are typically recorded in ad-hoc structured logs or passed through ephemeral message queues with no persistent causal record. ClockSync provides a drop-in replacement for this ad-hoc logging: each tool call becomes a cross-frame reference from the calling agent's ClockFrame to the called agent's response ClockFrame, transforming the ephemeral call graph into a persistent, cryptographically-ordered, auditable behavioral DAG. This integration requires only that each LLM-based sub-agent maintain its own ClockChain and participate in the ClockSync gossip network — a requirement compatible with the agent library deployment model described in Section 12.

8. Replayability

8.1 Definition of Deterministic Replayability

Deterministic Replayability is defined formally as follows: given a ClockVault V containing a complete, validated ClockFrame sequence $[F_0, F_1, \dots, F_n]$ for an agent A , and a conforming replay engine R , R can reconstruct agent A 's full behavioral history and re-execute any contiguous suffix $[F_k, \dots, F_n]$ (for any k in $[0, n]$) such that for all i in $[k, n]$, R 's re-execution produces a `post_state_hash` identical to $F_i.post_state_hash$, provided that the agent's logic is deterministic and the replay engine faithfully reproduces the `pre_state` from $F_k.pre_state_hash$.

This definition makes deterministic replayability conditional on two factors: (i) agent logic determinism (a design requirement for auditable agents), and (ii) availability of the agent's state at the anchor frame (provided by `pre_state_hash` and, in full replay mode, the complete state serialization stored in the ClockVault). The `post_state_hash` equality condition provides a verifiable correctness criterion: a replay that produces mismatching `post_state_hashes` is provably divergent from the original execution.

8.2 Replay Engine Architecture

The ClockChain Replay Engine is a purpose-built component that operates exclusively on ClockVault data and agent runtime interfaces. Its four principal components are:

- **Frame Iterator:** Traverses the ClockVault in sequence order, validating each frame's `parent_hash` and signature before presenting it to downstream components. Halts with a `ChainViolation` exception if any integrity check fails.
- **State Reconstructor:** Reconstructs the agent's internal state at any anchor frame by loading the serialized state snapshot associated with the anchor frame's `pre_state_hash`. For full replay, this is the `ChainRoot` state (the agent's initial state at genesis). For partial replay, this is the state at frame `k`'s `pre_state`, which must be stored in the ClockVault as a state snapshot (not all deployments store full state snapshots; this is a configurable ClockVault policy).
- **Action Re-invoker:** Re-executes the action declared in each frame's `action_payload` against the reconstructed agent runtime. For external side-effect actions (API calls, database writes), the Re-invoker operates in shadow mode by default: external calls are mocked using the `action_payload` and the recorded post-state, not actually executed. This prevents replay from producing real-world side effects.
- **Divergence Detector:** After each re-executed action, computes the SHA3-256 hash of the agent's updated state and compares it to the frame's

recorded `post_state_hash`. A mismatch raises a `DivergenceEvent`, logged with the frame index, the expected hash, and the computed hash.

Divergence Events are the forensic signal for non-deterministic behavior, external state mutation, or tampering.

8.3 Replayability Modes

Full Replay: The replay engine begins at the ChainRoot (`F_0`) and re-executes all frames in sequence from the agent's genesis state. Full Replay produces the most comprehensive behavioral reconstruction and is the appropriate mode for regulatory forensic analysis and incident investigation.

Partial Replay: The replay engine begins at a specified anchor frame `F_k`, loading the pre-state from the ClockVault's state snapshot at `k`. Partial Replay is used when the forensic question concerns a bounded time window, reducing computational cost when `n` is large. The correctness of Partial Replay depends on the availability of a stored state snapshot at `k`; without it, the engine must fall back to Full Replay from the genesis.

Counterfactual Replay: The replay engine begins at a specified anchor frame `F_k`, but modifies one or more `action_payload` values in the replay sequence before re-executing. Counterfactual Replay answers hypothetical questions — "what would have happened if the agent had made a different decision at frame `k`?" — by re-executing the modified decision and propagating forward to observe divergent outcomes. Counterfactual Replay is a powerful tool for policy analysis, regulatory stress-testing, and safety evaluation. The counterfactual replay trace is annotated as non-authoritative and must not be stored in the live ClockVault; it is ephemeral by design.

8.4 Boundary Conditions and Shadow Mode

External side effects — API calls, database writes, message publications — represent the principal boundary condition for deterministic replay. These actions are recorded in `ClockFrames` (their intent and outcome are captured in `action_payload` and `post_state_hash`), but their re-execution in a live environment would produce real-world consequences. The Replay Engine's shadow mode addresses this:

external action types are detected by the Action Re-invoker, which substitutes a mock response derived from the original `action_payload` rather than dispatching a live call. The mock response must faithfully reproduce the original response — if the original response is not stored in `action_payload` (only its hash is), shadow mode is limited to state verification rather than full re-execution.

For applications where full external-state fidelity is required during replay (e.g., regulatory forensic replay of a trading agent after a flash-crash event), ClockChain recommends that `action_payload` include the complete serialized response of all external calls at the time of the original execution, not merely a hash. This increases frame size (potentially significantly for large API responses) but enables complete, auditor-satisfying replay fidelity.

8.5 Forensic Application: Trading Agent Flash-Crash Replay

In the motivating fintech scenario of Section 2.4, a regulatory forensic replay proceeds as follows: (i) the regulator's Inspector Interface accesses the trading agent's ClockVault for the relevant time window; (ii) the Replay Engine is initialized in Full Replay mode, beginning at the genesis frame; (iii) the Frame Iterator validates the complete chain from genesis to the post-event frame; (iv) the Action Re-invoker re-executes each market observation, strategy decision, risk assessment, and order placement frame in sequence using shadow mode, with stored market data responses from `action_payload`; (v) the Divergence Detector confirms byte-identical `post_state_hashes` at every frame, establishing that the replay faithfully reproduces the original execution; (vi) the resulting replay trace is exported as a structured audit report. This process satisfies MiFID II Article 26 transaction reporting requirements with cryptographic-grade tamper-evidence.

9. Compliance Implications

9.1 Overview

ClockChain is designed with regulatory compliance as a first-class requirement, not a post-hoc consideration. The ClockFrame schema's `policy_refs` field enables each

behavioral commit to self-declare its compliance context; the AgentID anchoring mechanism provides unforgeable identity attribution; and the deterministic replayability guarantee provides the audit trail quality required by financial, data protection, and cybersecurity regulatory frameworks. This section maps ClockChain's technical properties to four major regulatory frameworks.

9.2 GDPR (EU General Data Protection Regulation) [4]

Article 22 — Right to Explanation: GDPR Article 22 grants individuals the right to an explanation of automated decisions that significantly affect them. ClockChain's frame sequence provides a machine-readable and human-interpretable decision trail: for any automated decision, the Inspector Interface can traverse the ClockChain from the DECIDE-type frame representing the decision backwards through its OBSERVE-type ancestor frames, reconstructing the precise sequence of observations and inferences that led to the decision. This trace is exportable as a structured explanation document, satisfying the "meaningful information" requirement of Recital 71.

Article 17 — Right to Erasure: ClockChain does not store personal data in ClockFrames — it stores behavioral hashes (pre_state_hash, post_state_hash) and action descriptors that reference personal data rather than containing it. This architectural choice means that GDPR erasure obligations apply to the underlying data stores (which the agent accesses), not to the ClockFrames themselves. When an erasure request is received, ClockChain implements *cryptographic erasure*: the encryption key used to encrypt any personal data embedded in action_payload fields is destroyed, rendering those payload fields permanently inaccessible while preserving the chain's structural integrity. Frame deletion is explicitly not supported; the chain's auditability must not be compromised by erasure requests.

Article 5(1)(e) — Storage Limitation: The policy_refs field can reference retention policy identifiers that bind each frame to a specified retention schedule. A compliance monitoring agent can traverse ClockVaults and identify frames whose retention period has expired, triggering cryptographic erasure of sensitive payload content while retaining the frame's structural fields (hashes and signatures) for audit continuity.

9.3 MiFID II (Markets in Financial Instruments Directive II) [3]

Article 27 — Best Execution Obligation: MiFID II Article 27 requires investment firms to take all sufficient steps to obtain the best possible result for clients when executing orders. For autonomous trading agents, demonstrating best execution requires showing the complete decision rationale: what market state was observed, what execution venues were considered, what cost and liquidity factors were evaluated, and why the executed venue was selected. ClockChain's OBSERVE and DECIDE-type frames capture precisely this information in a causally-ordered, cryptographically-sealed sequence, enabling best-execution justification reports to be auto-generated from the frame sequence.

Article 26 — Transaction Reporting Obligation: MiFID II Article 26 requires detailed transaction reports to be submitted to national competent authorities. ClockChain's ACT-type frames for order placements contain the `action_payload` fields necessary to populate MiFID II transaction report fields (instrument identifier, price, volume, timestamp, counterparty). A reporting agent — itself operating a ClockChain — can traverse trading agent ClockVaults and auto-generate Article 26 reports with cryptographic provenance, substantially reducing the operational cost and error rate of manual reporting workflows.

9.4 SOC 2 (Trust Services Criteria)

CC7.2 — Anomaly Detection: SOC 2 CC7.2 requires organizations to detect and respond to anomalies in system operation. ClockChain's Divergence Events — raised when a re-executed frame produces a `post_state_hash` that does not match the recorded value — constitute precisely the anomaly detection signal that CC7.2 requires. Divergence Events are themselves recorded as corrective commits in the ClockChain, creating an auditable record of anomaly detection and response.

CC6.1 — Logical Access Controls: SOC 2 CC6.1 requires controls over logical access to information assets. ClockChain's AgentID anchoring enforces non-repudiable access accountability: every action taken by every agent is cryptographically attributed to a specific AgentID, and the delegation chain records the full authority lineage of that attribution. The Inspector Interface's capability-

token access control (Section 4.2.5) ensures that even read access to ClockVault data is subject to attribution and logging.

9.5 NIST AI Risk Management Framework (AI RMF) [5]

GOVERN 1.7 — Accountability: NIST AI RMF GOVERN 1.7 requires that roles and responsibilities for AI risk management be documented and accountability be established. ClockChain's identity anchoring and delegation chain mechanisms provide cryptographic accountability at the agent level: for every behavioral commit, the responsible AgentID and its full delegation lineage are permanently recorded.

MANAGE 2.4 — Incident Response: NIST AI RMF MANAGE 2.4 requires organizations to have processes for identifying and responding to AI incidents. ClockChain's replayability capabilities — particularly Full Replay and Counterfactual Replay — provide the forensic infrastructure required for post-incident analysis. A Divergence Event raised during replay is a formal signal of an incident, triggering the organization's incident response process with a cryptographically-verified behavioral record as evidence.

9.6 Compliance-Grade Deployment

Tier 3 deployment (Section 4.3) — in which ClockVault Merkle roots are periodically anchored to an external, publicly-auditable hash registry — satisfies the evidentiary standard for tamper-evidence in financial and healthcare regulated environments. The anchoring process creates a cryptographically-verifiable chain of custody from any individual ClockFrame to a publicly-timestamped hash record, establishing that the frame existed and had its current content at the time of anchoring. This chain of custody is admissible as technical evidence in regulatory proceedings and judicial proceedings that accept digital tamper-evidence standards.

Key Compliance Insight:

ClockChain's structural separation of behavioral hashes from personal data — combined with cryptographic erasure semantics — means that GDPR compliance obligations are substantially simpler than for traditional audit logs that embed

personal data directly in log entries. The ClockChain behavioral record can be retained indefinitely for audit purposes without retaining personal data, satisfying simultaneously the long retention requirements of financial regulation and the storage limitation principles of GDPR.

10. Fintech Applications

10.1 Application 1: Autonomous Trading Agent Auditability

A trading agent executing equity, fixed income, or derivatives strategies on behalf of institutional clients commits each market observation (OBSERVE), strategy selection (DECIDE), risk check (DECIDE), and order placement (ACT) as ClockFrames. The `pre_state_hash` and `post_state_hash` fields capture the agent's position book state before and after each action. The `policy_refs` field on each order ACT frame references "MiFID2-Art26" and "MiFID2-Art27," enabling the reporting agent to auto-identify frames that contribute to regulatory reports. Following a flash-crash event or regulatory inquiry, the inspector performs a Full Replay from the relevant time window, producing a byte-identical re-execution trace as forensic evidence. MiFID II Article 26 transaction reports are auto-generated from ACT-type frames without manual data extraction.

Storage estimate: At approximately 2.1 KB per ClockFrame (accounting for payload, hashes, and signature overhead), an agent executing 1,000 behavioral commits per hour generates approximately 2.1 MB of ClockChain data per hour, or 50.4 MB per 24-hour trading day. This overhead is negligible compared to the terabytes of market data feeds typically ingested by trading infrastructure.

10.2 Application 2: Multi-Agent Credit Scoring Pipeline

A credit scoring pipeline comprising four specialized agents — data ingestion, feature engineering, scoring model inference, and decision agent — forms a ClockSync DAG with each stage's output frame referenced by the next stage's input

frame. The full causal chain from raw applicant data to credit decision is captured in the DAG: an auditor can traverse from the DECIDE-type frame representing "application denied" back through the scoring model's DECIDE frames, the feature engineering ACT frames, and the data ingestion OBSERVE frames, reconstructing the precise computation that produced the decision. This satisfies both FCRA (Fair Credit Reporting Act) adverse action explanation requirements and ECOA (Equal Credit Opportunity Act) non-discrimination audit requirements. The ClockSync DAG replaces fragile, manually-maintained data lineage documentation with a cryptographically-ordered, automatically-generated causal record.

10.3 Application 3: Cross-Border Fraud Detection Agent Coordination

Fraud detection agents operating across different jurisdictions (e.g., EU and US) must coordinate behavioral signals without sharing raw customer data across borders — a requirement imposed by GDPR's data transfer restrictions. ClockSync enables GDPR-compliant cross-border coordination: agents share ClockFrame hashes (behavioral attestations) rather than raw data. An EU fraud agent can declare a causal dependency on a US fraud agent's committed OBSERVE frame — attesting that it is incorporating the US agent's behavioral conclusion into its own decision — without the US agent transmitting the underlying customer data across the border. The behavioral DAG captures the coordination with cryptographic integrity while maintaining data localization compliance.

10.4 Application 4: Regulatory Reporting Agent

A specialized regulatory reporting agent — one whose sole operational function is to traverse the ClockVaults of other agents and generate structured regulatory reports — is itself a ClockChain participant. The reporting agent commits OBSERVE frames (reading target agent ClockVaults), DECIDE frames (classifying frames by regulatory report type), and ACT frames (generating and submitting reports). The reporting agent's own ClockChain provides a cryptographically-sealed audit trail of the reporting process itself: which ClockVaults were read, which frames were included in which reports, and when each report was submitted. Report integrity is

anchored in the reporting agent's ClockChain, enabling regulators to verify that reports were generated from the authentic, unmodified ClockVault data.

10.5 Application 5: Smart Contract Checkpoint Integration

For applications requiring public, adversarially-tamper-resistant auditability, ClockChain's Tier 3 deployment can anchor frame Merkle roots to an EVM-compatible smart contract [8, 16]. A checkpointing agent periodically computes the Merkle root of all ClockFrames committed since the last checkpoint and submits a transaction to the smart contract recording this root. Any third party can subsequently verify that any given ClockFrame was included in a checkpoint by presenting a Merkle audit path — achievable with $O(\log n)$ space and verifiable with $O(\log n)$ hash operations. This architecture achieves the auditability of ClockChain with the public tamper-resistance of a blockchain, without requiring on-chain execution of agent logic or on-chain storage of individual frames.

11. Comparison to Blockchain

11.1 Architectural Distinctions

ClockChain and blockchain ledgers share the structural element of a cryptographically-chained, append-only sequence of records, but differ fundamentally in design philosophy, operational scope, and semantic model. Understanding these distinctions is essential for practitioners evaluating which technology is appropriate for a given behavioral auditability requirement.

The most fundamental distinction is scope: blockchain is *network-global first* — its security properties derive from global distribution and consensus among mutually-distrusting parties. ClockChain is *agent-local first* — its security properties derive from cryptographic integrity of a per-agent chain, with federation as an optional extension. This design difference produces cascading consequences across all other dimensions of comparison.

11.2 Comparative Analysis Table

Dimension	ClockChain	Public Blockchain (e.g., Ethereum)	Private/Permissioned Blockchain
Identity Model	Behavior-linked, genesis-frame-derived AgentID; persistent across restarts; delegation-chain-aware	Key-pair address; no behavioral semantics; no persistence mechanism across key rotations	Permissioned membership; often certificate-based; no behavioral linkage
Consensus Mechanism	None required (Tier 1-2); optional quorum for federated settlement (ClockSync)	Proof of Work [7] or Proof of Stake; global validator set required	PBFT or variants [23]; permissioned validator set; lower overhead than public
Finality	Deterministic per-agent finality on append; federated settlement adds ~2-5ms; no probabilistic delay	Probabilistic; typically 6-64 block confirmations (~1-13 minutes for Ethereum)	Near-deterministic; typically 1-3 seconds for PBFT-class protocols
Throughput	O(1) per agent; limited by local I/O (thousands of commits/second); linear scale with agent count	~10-30 TPS (Bitcoin); ~15-100 TPS (Ethereum L1); throughput shared across all users	~1,000-10,000 TPS typical; shared across permissioned network participants
Agent-Native Semantics	Yes — ClockFrame action_type enum, pre/post state hashes, capability manifests, policy_refs are first-class	No — transactions encode asset transfers or contract calls; agent concepts require application-layer mapping	No — same as public blockchain; smart contract abstraction required for agent semantics
Compliance Grade	Tier 3: evidentiary-grade with external hash anchoring; policy_refs enable per-frame compliance tagging	High tamper-resistance; but regulatory mapping requires significant application-layer work	Operator-dependent; compliance frameworks vary by implementation
Deployment Complexity	Library (Tier 1-2); no network infrastructure required for local deployment; federation optional	High — requires full node or reliance on third-party RPC providers; gas cost management; network membership	Medium — requires permissioned network setup, certificate management, validator configuration

Dimension	ClockChain	Public Blockchain (e.g., Ethereum)	Private/ Permissioned Blockchain
Replayability	Yes — Full, Partial, and Counterfactual replay; byte-identical deterministic re-execution	Partial — transaction replay possible; state counterfactuals require forked testnet; not operationally supported	Partial — same limitations as public blockchain; some private chains support state snapshots

Table 2. Comparative analysis of ClockChain versus blockchain architectures across eight dimensions.

11.3 When to Choose Blockchain

Blockchain ledgers remain the appropriate choice in scenarios characterized by: (i) *adversarial multi-party environments* where no single operator can be trusted to maintain ledger integrity and mutual distrust between all parties is the operating assumption; (ii) *asset settlement requiring network-wide finality*, where the transfer of ownership must be recognized simultaneously by all network participants; (iii) *open participation*, where the set of participants is not known in advance and permissioned enrollment is not operationally feasible; and (iv) *programmable settlement logic*, where smart contract execution is required to enforce settlement conditions automatically.

11.4 When to Choose ClockChain

ClockChain is the appropriate choice in scenarios characterized by: (i) *enterprise or federated agent deployments* where a trusted operator or federation of known operators governs the validator set; (ii) *behavioral auditability over asset settlement semantics*, where the goal is to record what agents did rather than to settle asset ownership; (iii) *high-frequency behavioral commit requirements*, where the throughput and latency constraints of blockchain are operationally prohibitive; (iv) *regulatory compliance*, where per-frame compliance tagging, cryptographic erasure semantics, and deterministic replayability are required; and (v) *low deployment complexity*, where the system must be deployable as a library without network infrastructure.

11.5 Hybrid Architecture

The two technologies are not mutually exclusive. A hybrid architecture — ClockChain Tier 3 deployment — combines the behavioral semantics and throughput of ClockChain with the adversarial tamper-resistance of public blockchain. In this architecture, ClockChain operates as the primary behavioral ledger, handling high-frequency commits with deterministic finality. Periodically (e.g., every 15 minutes or every 10,000 frames), the ClockVault's Merkle root is submitted to a public smart contract [16] as a hash checkpoint. The on-chain checkpoint serves as a cryptographic notarization: any subsequent dispute about the ClockVault's contents can be resolved by verifying the Merkle audit path against the on-chain root. This hybrid achieves the behavioral auditability of ClockChain with the public, adversarially-tamper-resistant record of blockchain, at a fraction of the cost of recording all frames on-chain.

12. Conclusion

This paper has introduced ClockChain, a deterministic behavioral ledger designed to serve as purpose-built infrastructure for the emerging paradigm of auditable autonomous agents. ClockChain addresses a structural gap in the landscape of distributed systems and agent infrastructure: the absence of a unified primitive that combines persistent agent identity, causally-ordered behavioral commits, cryptographic tamper-evidence, deterministic replayability, and compliance-grade auditability in a single, deployable protocol.

The paper has presented: a formal model of agent identity anchoring via cryptographically-derived AgentIDs bound to genesis ClockFrames; a deterministic two-phase commit model producing cryptographically-chained ClockFrames with byte-reproducible replay guarantees; the ClockSync gossip protocol for multi-agent causal synchronization forming a globally-consistent behavioral DAG; three replayability modes — full, partial, and counterfactual — with formal correctness criteria; a structured mapping to GDPR, MiFID II, SOC 2, and the NIST AI RMF; and analysis of five high-value fintech deployment scenarios.

The key insight that motivates ClockChain's design is that behavioral auditability requires a purpose-built primitive, not a repurposed asset ledger. Blockchain technology addresses asset transfer semantics in adversarial multi-party environments; ClockChain addresses behavioral commitment semantics in federated or enterprise agent environments. The two are complementary, and the Tier 3 hybrid architecture described in Section 11.5 demonstrates how they can be combined to achieve the full spectrum of tamper-resistance requirements.

ClockChain is designed to be deployable as a software library — not a network — placing Tier 1 behavioral auditability within reach of any agent developer with no infrastructure investment. The authors believe that this design choice is essential for broad adoption: auditability must be the path of least resistance, not an expensive compliance retrofit. The broader vision is ClockChain as foundational infrastructure for the auditable agent paradigm — a world in which every autonomous agent, by default, maintains a cryptographically-sealed, causally-ordered, replayable behavioral record that satisfies the evidentiary requirements of any regulatory framework, and in which auditability is as natural and automatic as logging is today.

13. Future Work

Zero-Knowledge Compliance Proofs. A compelling extension of ClockChain is the integration of zero-knowledge proof systems [26] to enable agents to prove regulatory compliance without revealing frame contents. For example, an agent could generate a ZK proof that its DECIDE frames satisfy the MiFID II best-execution criteria without revealing the specific market state observations or strategy parameters that drove the decision. This would enable privacy-preserving compliance certification — a significant advance for multi-party coordination scenarios where behavioral data is commercially sensitive.

Sub-Action Granularity for LLM Agents. Current ClockFrame semantics operate at the action level — one frame per atomic agent action. For LLM-based agents, where a single "action" may involve dozens of token generation steps, tool calls, and intermediate reasoning traces, sub-action granularity may be required for full

behavioral transparency. Future work will explore integrating ClockChain with token-level generation traces, enabling ClockFrames to reference specific token spans within a generation trace as behavioral sub-commits.

Formal Verification of ClockSync. The ClockSync protocol's correctness properties — causal consistency, concurrent commit detection, and conflict resolution — are stated informally in this paper and supported by analytical argument. Formal verification using TLA+ [27] or Coq would provide machine-checkable correctness proofs, substantially strengthening the confidence of regulatory bodies and safety engineers in ClockSync's guarantees. This is a high-priority direction for future work.

Hardware Security Module Integration. The security of ClockChain's identity anchoring and frame signing depends critically on the protection of the agent's Ed25519 private key. Integration with Hardware Security Modules (HSMs) — including TPM 2.0 chips on standard server hardware — would provide hardware-rooted key protection, ensuring that key compromise requires physical access to the HSM and is detectable via attestation. This is essential for Tier 3 regulatory-grade deployments.

Federated ClockVault as a Managed Cloud Service. Lowering the operational barrier to Tier 2 and Tier 3 deployments would be substantially advanced by offering a managed federated ClockVault service — analogous to managed database services — that handles validator set management, Merkle root checkpointing, Inspector Interface hosting, and compliance report generation as a cloud-native service. The authors plan to prototype such a service as part of the ClockChain open-source initiative.

Standardization Path. Long-term, the ClockFrame schema and ClockSync protocol should be proposed as IETF or W3C standards — analogous to the W3C DID specification [11] or the IETF RFC 3161 trusted timestamping protocol — to enable interoperability across organizations, jurisdictions, and technology stacks. Standardization would allow regulatory bodies to reference ClockChain-compliant behavioral records in formal requirements, creating a stable compliance target for the agent infrastructure industry.

References

7. **[1]** S. Park, K. Lee, and J. Cho, "Autonomous AI Agents in Enterprise Workflows: A Survey of Deployment Architectures and Risk Profiles," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–38, 2025.
8. **[2]** A. Guo, R. Patel, and M. Chen, "Multi-Agent LLM Orchestration: Patterns, Pitfalls, and Accountability Gaps," in *Proc. NeurIPS Workshop on Agentic AI Systems*, New Orleans, LA, Dec. 2025, pp. 12–24.
9. **[3]** European Parliament, "Directive 2014/65/EU on Markets in Financial Instruments (MiFID II)," *Official Journal of the European Union*, L 173, Jun. 2014. Consolidated version 2018.
10. **[4]** European Parliament, "Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)," *Official Journal of the European Union*, L 119, Apr. 2016.
11. **[5]** National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Gaithersburg, MD, Jan. 2023.
12. **[6]** N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA: MIT Press, 2011.
13. **[7]** S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Whitepaper, Oct. 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
14. **[8]** V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum Whitepaper, 2014. [Online]. Available: <https://ethereum.org/whitepaper>
15. **[9]** B. Schneier and J. Kelsey, "Secure Audit Logs to Support Computer Forensics," *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, May 1999.
16. **[10]** L. Moreau, P. Missier, K. Belhajjame, R. B'Far, J. Cheney, S. Coppens, S. Cresswell, Y. Gil, P. Groth, G. Klyne, T. Lebo, J. McCusker, S. Miles, J. Myers, S. Sahoo, and C. Tilmes, "PROV-DM: The PROV Data Model," W3C Recommendation, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>

17. **[11]** M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022. [Online]. Available: <https://www.w3.org/TR/did-core/>
18. **[12]** L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, Jul. 1982.
19. **[13]** L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.
20. **[14]** C. J. Fidge, "Timestamps in Message-Passing Systems That Preserve the Partial Ordering," in *Proc. 11th Australian Computer Science Conference (ACSC-11)*, Brisbane, Australia, Feb. 1988, vol. 10, no. 1, pp. 56–66.
21. **[15]** S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone, "Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases," in *Proc. 18th International Conference on Principles of Distributed Systems (OPODIS 2014)*, Cortina d'Ampezzo, Italy, Dec. 2014, pp. 17–32.
22. **[16]** G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, Berlin Version, 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
23. **[17]** M. Wooldridge and N. R. Jennings, "Intelligent Agents: Theory and Practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, Jun. 1995.
24. **[18]** L. Chen, H. Wang, Y. Zhang, and K. Park, "TraceAgent: Runtime Auditability for Large Language Model Tool-Call Pipelines," in *Proc. ACM Conference on Computer and Communications Security (CCS '25)*, Salt Lake City, UT, Nov. 2025, pp. 441–456.
25. **[19]** J. Wei, T. Brown, B. Askeel, A. Amodei, and D. Ramage, "Behavioral Transparency in Deployed LLM Agents: A Framework for Runtime Observability," *arXiv preprint arXiv:2501.08834*, Jan. 2025.
26. **[20]** O. Vaziry, C. Anand, and P. Hernandez, "Economic Identity Anchoring for Agent-to-Agent (A2A) Financial Protocols," in *Proc. ACM Workshop on Financial Cryptography and Data Security (FC '25)*, Raleigh, NC, Mar. 2025, pp. 89–104.

27. **[21]** D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-Speed High-Security Signatures," in *Proc. 13th International Workshop on Cryptographic Hardware and Embedded Systems (CHES 2011)*, Nara, Japan, Sep. 2011, Lecture Notes in Computer Science, vol. 6917, pp. 124–142.
28. **[22]** H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," IETF RFC 5869, May 2010.
29. **[23]** M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance and Proactive Recovery," *ACM Transactions on Computer Systems*, vol. 20, no. 4, pp. 398–461, Nov. 2002.
30. **[24]** National Institute of Standards and Technology, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," FIPS PUB 202, Gaithersburg, MD, Aug. 2015.
31. **[25]** M. Ahamad, G. Neiger, J. E. Burns, P. Kohli, and P. W. Hutto, "Causal Memory: Definitions, Implementation, and Programming," *Distributed Computing*, vol. 9, no. 1, pp. 37–49, Mar. 1995.
32. **[26]** S. Goldwasser, S. Micali, and C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, Feb. 1989.
33. **[27]** L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston, MA: Addison-Wesley, 2002.